

Magic Film

Software de MF (Magic Film)

Manual Técnico

Versión 1.0 - mayo del 2026

Luis Enrique Riascos Palacios

Yennifer Salas Ibarra

Michael Camilo Marín Hernández

Corporación Universitaria Autónoma Del Cauca

Ingeniería De Software Y Computación

Docente: Ana María Caviedes

Popayán - Cauca

Ficha del documento

Fecha	Revisión	Autores	Versión
28/05/2026	Docente	Michael Hernández Yennifer Salas Luis Riascos	1.0

1. Información General del Proyecto

- **Nombre del proyecto:** Magic Film — Plataforma web de análisis cinematográfico adaptativo con inteligencia artificial.
- **Integrantes:** Luis Enrique Riascos Palacios, Yennifer Salas Ibarra, Michael Camilo Marín Hernández.
- **Docente:** Ana María Caviedes.
- **Versión del sistema:** 1.0 (Mayo 2026).

Tecnologías utilizadas

Capa	Tecnología	Versión / Detalle
Frontend	HTML5, CSS3, JavaScript ES6+	Vanilla (sin frameworks)
Backend	Node.js + Express	Node 18+ / Express 4.18
Base de datos	MySQL	8.0+ con mysql2 driver
IA (Análisis)	Groq (LLaMA 3.3 70B)	API REST — llama-3.3-70b-versatile
Datos de películas	TMDB API v3	API Key con rate-limiting gestionado
Autenticación	JWT + bcrypt + Google OAuth 2.0	jsonwebtoken 9.x, bcryptjs 2.4, google-auth-library 10.x
Email	Nodemailer (Gmail SMTP)	nodemailer 8.x
Dev Server	Nodemon	3.x

2. Descripción General

Objetivo del sistema

Desarrollar una plataforma web que permita a los usuarios buscar películas y obtener análisis cinematográficos generados automáticamente mediante inteligencia artificial, adaptados al tipo de película (entretenimiento o profundo), presentados de forma estructurada por capas temáticas y personalizados según el perfil del usuario.

Problema que resuelve

Las plataformas actuales de referencia cinematográfica (IMDb, Letterboxd, Filmaffinity) ofrecen calificaciones numéricas y reseñas escritas por otros usuarios, sin un análisis estructurado, consistente ni adaptado al tipo de película. No existe diferenciación automática entre producciones comerciales y cine de autor, lo que resulta en contenido genérico que no ayuda al usuario a comprender el significado real de lo que vio.

Magic Film resuelve esto generando análisis cinematográficos adaptativos con IA, clasificando automáticamente cada película y produciendo contenido organizado en capas temáticas (narrativa, simbolismo, contexto, técnica) sin depender de reseñas de terceros.

Alcance del proyecto

- Registro e inicio de sesión con verificación por email y Google OAuth.
- Onboarding con selección de nivel cinéfilo y 3 géneros favoritos (incluyendo K-Drama).
- Búsqueda de películas y series conectada a TMDb en tiempo real.
- Clasificación automática del tipo de película (entretenimiento / profundo).
- Generación de análisis cinematográfico adaptativo mediante Groq (LLaMA 3.3 70B).
- Visualización del análisis organizado en capas navegables con pestañas.
- Sistema de favoritos y historial de búsquedas por usuario.
- Perfil de usuario con foto, biografía, géneros favoritos y nivel cinéfilo.
- Sección de comunidad para explorar perfiles de otros cinéfilos.
- Panel de administración con gestión de usuarios, películas, análisis y estadísticas.
- Formulario de contacto funcional con envío de email a magicfilm001@gmail.com.
- 14 pantallas, 45+ endpoints API, 10 tablas en base de datos.

3. Arquitectura del Sistema

Descripción de la arquitectura

Magic Film utiliza una arquitectura cliente-servidor de tres capas con patrón MVC (Modelo-Vista-Controlador) en el backend:

- Capa de presentación (Frontend): HTML5 + CSS3 + JavaScript ES6+ vanilla, servido como archivos estáticos por Express. Responsable de la interfaz de usuario, animaciones cinematográficas y comunicación con la API REST.
- Capa de lógica de negocio (Backend): Node.js con Express. Implementa controladores, servicios, modelos, middlewares y rutas. Gestiona autenticación, clasificación de películas, generación de análisis con IA y comunicación con APIs externas.
- Capa de datos: MySQL 8.0+ con 10 tablas normalizadas. Almacena usuarios, películas, análisis generados, favoritos, historial y logs de IA.

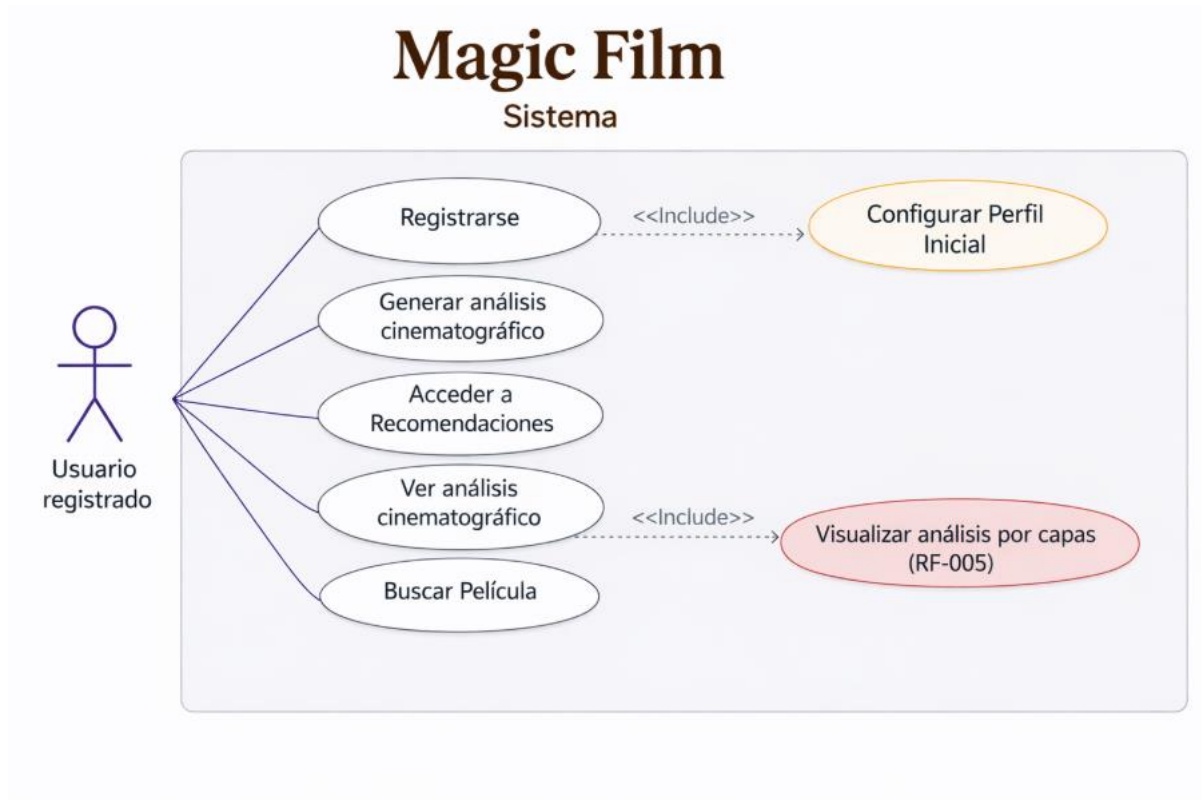
Diagrama arquitectónico

El flujo principal del sistema sigue esta secuencia:

1. El usuario busca una película desde el frontend.
2. El backend consulta TMDB API para obtener información de la película.
3. El clasificadorService analiza géneros, keywords y sinopsis para determinar el tipo (entretenimiento / profundo).
4. Si no existe análisis previo, el geminiService envía un prompt estructurado a Groq API.
5. Groq genera el análisis por capas, que se almacena en MySQL.
6. El frontend presenta el análisis organizado en pestañas navegables.

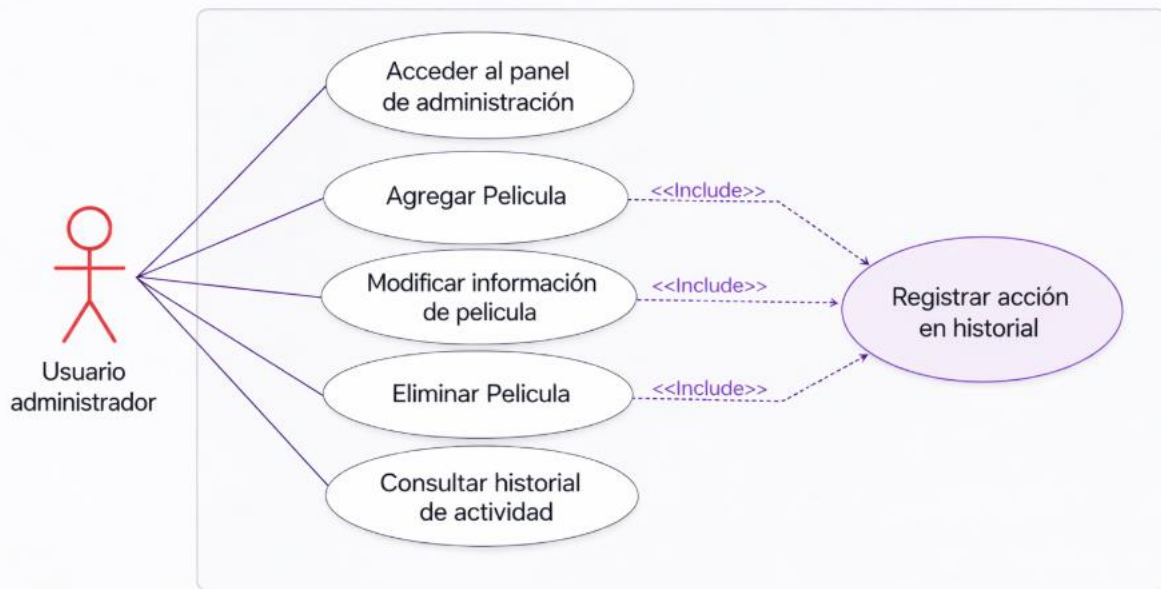
Diagrama De Casos De Uso

?



Magic Film

Sistema



Modelo Entidad Relación

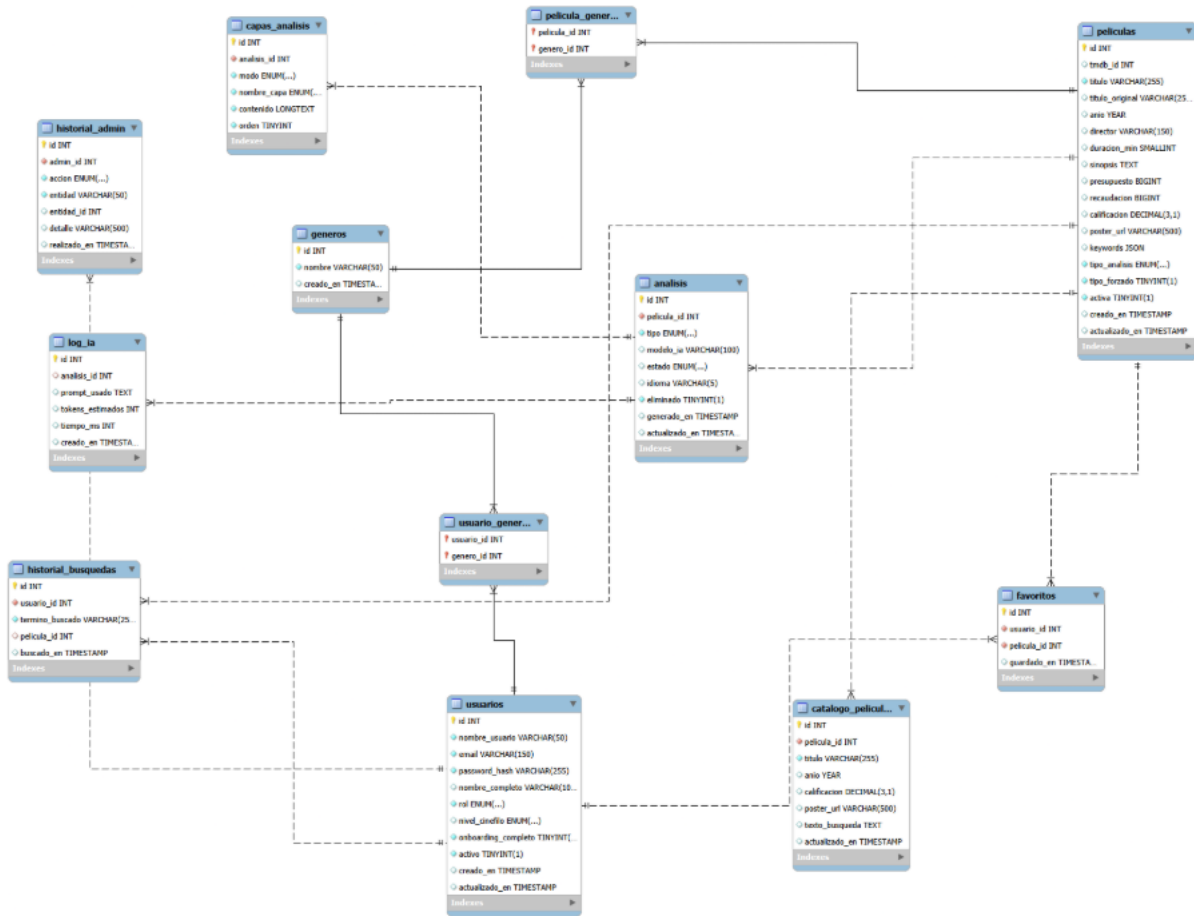


Diagrama de clases

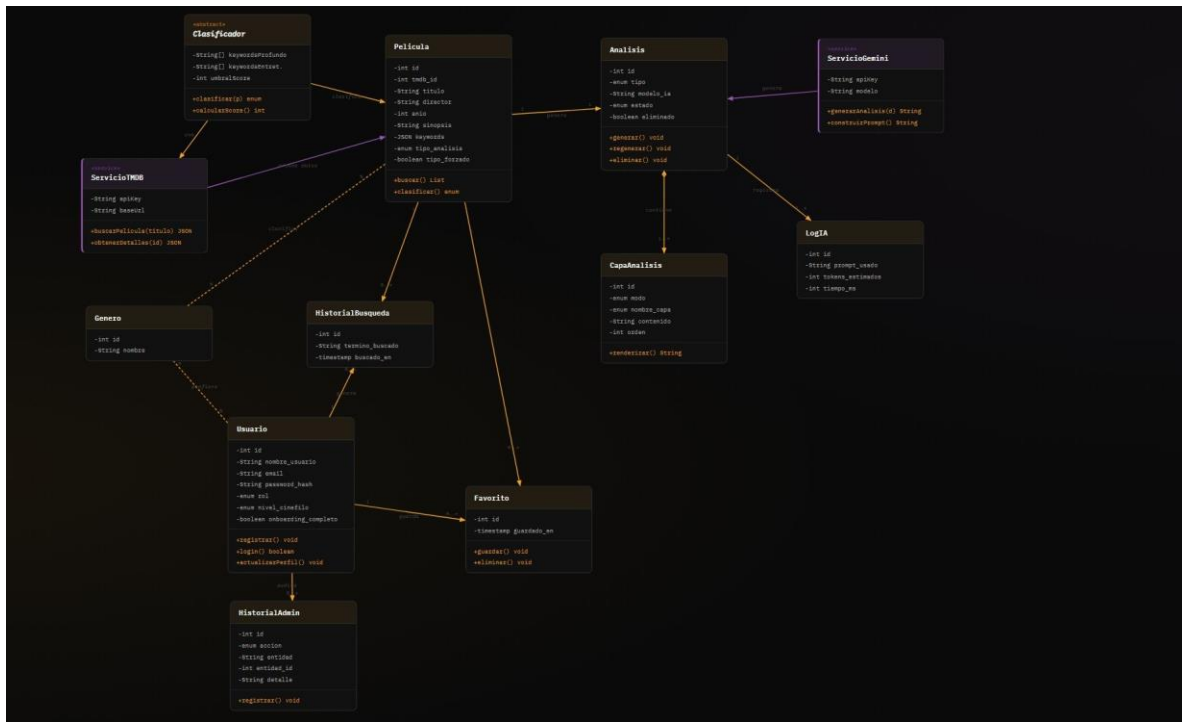


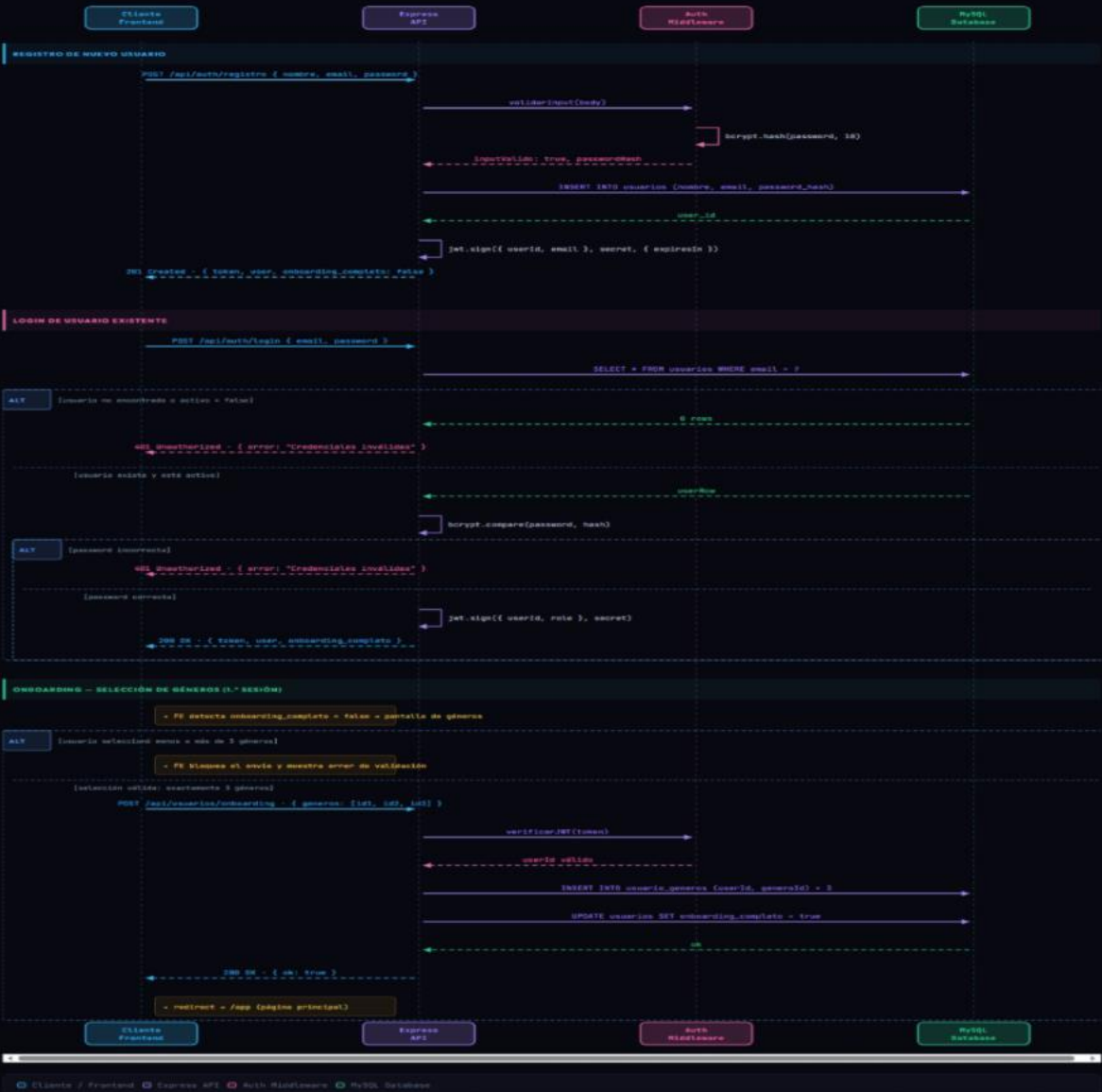
Diagrama de componentes



Diagrama de secuencia

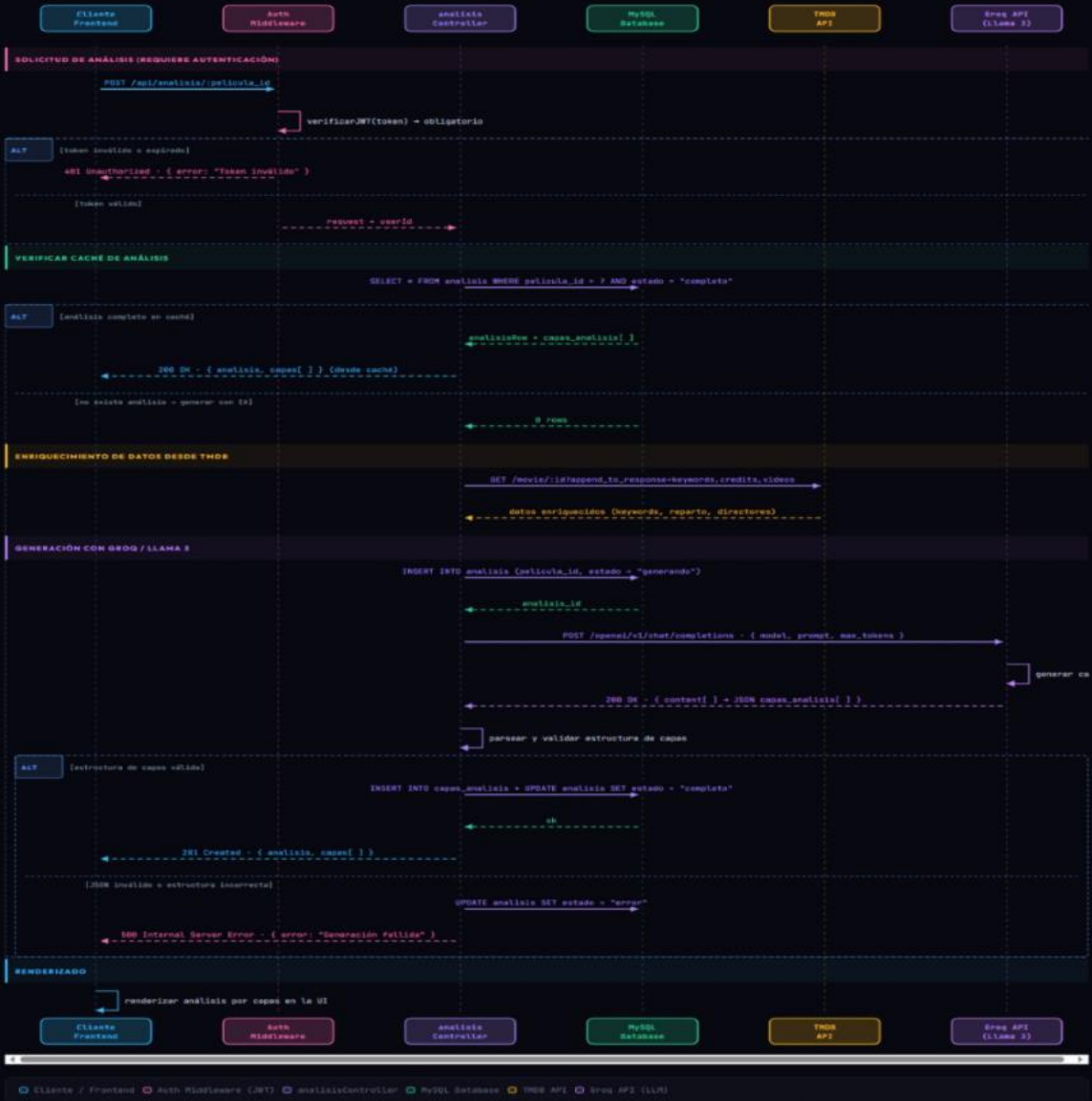
Flujo 1 — Registro de Usuario, Login y Onboarding

Proceso desde el registro inicial hasta la selección obligatoria de 3 géneros favoritos. El JWT se genera inmediatamente tras el registro y el login.



Flujo 4 — Generación de Análisis con IA (Grok / Llama 3)

Interacción de datos desde TMDB, generación con Groq LLM, validación de capas y lógica de cache. Requiere JWT válido.



Explicación de componentes

Componente	Archivo(s)	Responsabilidad
Servidor	server.js	Punto de entrada. Configura Express, middlewares, rutas y sirve el frontend estático.
Configuración	config/env.js, config/db.js	Variables de entorno y pool de conexiones MySQL.
Controladores	controllers/*.js (5 archivos)	Lógica de negocio: auth, películas, análisis, usuarios, admin.
Modelos	models/*.js (7 archivos)	Acceso a datos: Usuario, Pelicula, Análisis, Favorito, Genero, etc.
Servicios	services/*.js (4 archivos)	Integración con TMDB, Groq (IA), clasificador de películas, email.
Middlewares	middlewares/*.js (3 archivos)	Autenticación JWT, verificación admin, validación de input.
Rutas	routes/*.js (5 archivos)	Endpoints REST: /api/auth, /api/peliculas, /api/analisis, /api/usuarios, /api/admin.
Utilidades	utils/*.js (2 archivos)	Manejo estandarizado de respuestas y errores.

4. Requerimientos Técnicos

Hardware requerido

Componente	Mínimo	Recomendado
Procesador	Dual-core 2.0 GHz	Quad-core 2.5 GHz+
Memoria RAM	4 GB	8 GB
Almacenamiento	500 MB libres	1 GB libres
Red	Conexión a Internet estable	Banda ancha 10 Mbps+

Software requerido

Software	Versión	Propósito
Node.js	18.x o superior	Entorno de ejecución del backend
npm	9.x o superior	Gestión de dependencias
MySQL Server	8.0 o superior	Base de datos relacional
Git	2.x	Control de versiones
Navegador web	Chrome 90+, Firefox 90+, Edge 90+	Acceso al frontend

Dependencias (package.json)

Paquete	Versión	Función
express	^4.18.0	Framework web para la API REST
mysql2	^3.0.0	Driver MySQL con soporte para promesas
jsonwebtoken	^9.0.0	Generación y verificación de tokens JWT
bcryptjs	^2.4.3	Hashing seguro de contraseñas
cors	^2.8.5	Habilitación de Cross-Origin Resource Sharing
dotenv	^16.0.0	Carga de variables de entorno desde .env
nodemailer	^8.0.5	Envío de correos electrónicos vía SMTP
google-auth-library	^10.6.2	Verificación de tokens de Google OAuth
nodemon (dev)	^3.0.0	Recarga automática en desarrollo

5. Instalación y Configuración

Pasos de instalación

1. Clonar el repositorio:

```
git clone <Magic Film>
```

2. Instalar dependencias del backend:

```
cd backend && npm install
```

3. Crear la base de datos ejecutando el schema:

```
mysql -u root -p < database/schema.sql
```

4. Configurar las variables de entorno (ver sección siguiente).

5. Iniciar el servidor:

```
npm run dev      # Desarrollo (con nodemon)
```

```
npm start       # Producción
```

6. Acceder a la aplicación en <http://localhost:3000>

Configuración del entorno

Crear un archivo `.env` en la carpeta `backend/` con las siguientes variables:

Variables de entorno

Variable	Descripción	Ejemplo
PORT	Puerto del servidor	3000
NODE_ENV	Ambiente de ejecución	development
DB_HOST	Host de MySQL	localhost
DB_PORT	Puerto de MySQL	3307
DB_USER	Usuario de MySQL	root
DB_PASSWORD	Contraseña de MySQL	(tu contraseña)
DB_NAME	Nombre de la base de datos	magic_filmv01
JWT_SECRET	Clave secreta para JWT	(clave segura aleatoria)
JWT_EXPIRES_IN	Tiempo de expiración del token	7d
EMAIL_USER	Correo Gmail para envío	magicfilm001@gmail.com
EMAIL_PASS	App password de Gmail	(contraseña de aplicación)
GOOGLE_CLIENT_ID	Client ID de Google OAuth	(ID del proyecto en Google Cloud)
APP_URL	URL pública de la aplicación	http://localhost:3000

TMDB_API_KEY	API Key de TMDB	(clave de themoviedb.org)
GROQ_API_KEY	API Key de Groq	(clave de console.groq.com)
GROQ_MODEL	Modelo de Groq a usar	llama-3.3-70b-versatile
YOUTUBE_API_KEY	API Key de YouTube	(clave de Google Cloud)

6. Base de Datos

Modelo entidad relación

El modelo entidad-relación completo se encuentra en el documento "Documentación Magic Film" (Documentacion.Magic-Film.pdf, página 28). La base de datos magic_filmv01 utiliza MySQL 8.0+ con charset utf8mb4 y consta de 10 tablas principales más 1 tabla de códigos de verificación.

Diccionario de datos

Tabla: generos

Columna	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador único del género
nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre del género cinematográfico
creado_en	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación

Tabla: usuarios

Columna	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador único
nombre_usuario	VARCHAR(50)	NOT NULL, UNIQUE	Username único
email	VARCHAR(150)	NOT NULL, UNIQUE	Correo electrónico
password_hash	VARCHAR(255)	DEFAULT NULL	Hash bcrypt de la contraseña
nombre_completo	VARCHAR(100)		Nombre completo del usuario
rol	ENUM	NOT NULL, DEFAULT	invitado / registrado /

		registrado	admin
nivel_cinefilo	ENUM	DEFAULT NULL	explorador / entusiasta / experto
onboarding_completo	BOOLEAN	NOT NULL, DEFAULT FALSE	Si completó el onboarding
activo	BOOLEAN	NOT NULL, DEFAULT TRUE	Estado de la cuenta
foto_perfil	MEDIUMTEXT	DEFAULT NULL	Foto en base64
bio	VARCHAR(300)	DEFAULT NULL	Biografía del usuario
email_verificado	BOOLEAN	NOT NULL, DEFAULT FALSE	Estado de verificación
google_id	VARCHAR(100)	DEFAULT NULL	ID de Google OAuth
creado_en	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de registro

Tabla: peliculas

Columna	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador interno
tmdb_id	INT UNSIGNED	UNIQUE	ID en TMDB
titulo	VARCHAR(255)	NOT NULL	Título en español
titulo_original	VARCHAR(255)		Título original
anio	YEAR		Año de estreno
director	VARCHAR(150)		Director principal
duracion_min	SMALLINT UNSIGNED		Duración en minutos
sinopsis	TEXT		Sinopsis de TMDB
calificacion	DECIMAL(3,1)	CHECK 0-10	Calificación promedio
poster_url	VARCHAR(500)		URL del póster
backdrop_url	VARCHAR(500)		URL de imagen de fondo
keywords	JSON		Palabras clave de TMDB
tipo_analisis	ENUM	DEFAULT entretenimiento	entretenimiento / profundo
media_type	ENUM	DEFAULT movie	movie / tv
reparto	JSON		Actores principales
similares	JSON		Películas similares

Tabla: analisis

Columna	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador del análisis
pelicula_id	INT UNSIGNED	NOT NULL, UNIQUE, FK	Referencia a películas.id
tipo	ENUM	NOT NULL	entretenimiento / profundo
modelo_ia	VARCHAR(100)	DEFAULT llama-3.3-70b-versatile	Modelo de IA utilizado
estado	ENUM	DEFAULT pendiente	pendiente / generando / completo / error
idioma	VARCHAR(5)	DEFAULT es	Idioma del análisis
eliminado	BOOLEAN	NOT NULL, DEFAULT FALSE	Borrado lógico

Tabla: capas_analisis

Columna	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador de la capa
analisis_id	INT UNSIGNED	NOT NULL, FK	Referencia a analisis.id
modo	ENUM	NOT NULL	entretenimiento / profundo
nombre_capa	ENUM	NOT NULL	resumen, momentos, easter_eggs, curiosidades, narrativa, simbolismo, contexto, tecnica, conclusion
contenido	LONGTEXT	NOT NULL	Contenido HTML del análisis
orden	TINYINT UNSIGNED	DEFAULT 1	Orden de visualización

Otras tablas del sistema:

Tabla	Descripción	Relaciones principales
usuario_generos	Géneros favoritos del usuario (N:M)	FK a usuarios.id
pelicula_generos	Géneros de cada película (N:M)	FK a peliculas.id, generos.id
favoritos	Películas marcadas como favoritas	FK a usuarios.id, peliculas.id
historial_busquedas	Términos buscados por el usuario	FK a usuarios.id, peliculas.id
historial_admin	Acciones administrativas registradas	FK a usuarios.id (admin)
log_ia	Registro de llamadas a la API de Groq	FK a analisis.id
codigos_verificacion	Códigos de 6 dígitos para registro	Índice por email
catalogo_peliculas	Caché de búsqueda del catálogo	FK a peliculas.id

Scripts necesarios

El script principal de creación de la base de datos se encuentra en:

`database/schema.sql`

Este script crea la base de datos, todas las tablas, índices, datos iniciales de géneros (incluyendo K-Drama) y el usuario administrador por defecto.

Para datos de prueba adicionales:

`database/seeds/data_prueba.sql`

7. Estructura del Código

Organización de carpetas

```
MagicFilm/
├── backend/
```

```

├── config/           # Configuración (env.js, db.js)
├── controllers/      # Controladores (auth, peliculas, analisis, usuarios, admin)
├── middlewares/      # Autenticación JWT, admin, validación
├── models/           # Modelos de datos (Usuario, Pelicula, Analisis, etc.)
├── routes/           # Definición de endpoints REST
├── services/         # Servicios: TMDB, Groq, clasificador, email
├── utils/            # Helpers de respuesta y errores
├── server.js         # Punto de entrada del backend
├── package.json
├── frontend/
│   ├── assets/       # Favicon, imágenes
│   ├── css/
│   │   ├── components/ # Botones, cards, forms, modal, navbar, tabs
│   │   ├── pages/      # Estilos por página (styles.css, login.css, admin.css, etc.)
│   │   ├── global.css  # Variables y reset global
│   │   └── variables.css
│   ├── js/
│   │   ├── api/        # Clientes API (auth, peliculas, analisis)
│   │   ├── components/ # Componentes JS (loader, modal, navbar, tabs)
│   │   └── pages/      # Lógica por página (busqueda.js, analisis.js, perfil.js, etc.)
│   └── pages/         # 14 páginas HTML
├── database/
│   ├── schema.sql     # Script completo de creación de BD
│   └── seeds/         # Datos de prueba
└── docs/              # Documentación del proyecto

```

Descripción de módulos

Backend — Controladores:

Controlador	Endpoints principales	Funcionalidad
authController	/api/auth/*	Registro, login, Google OAuth, verificación email, forgot/reset password
peliculasController	/api/peliculas/*	Búsqueda multi-media, populares, por género (inc. K-Drama), detalles, YouTube
analisisController	/api/analisis/*	Generación y obtención de análisis cinematográficos adaptativos
usuariosController	/api/usuarios/*	Perfil, onboarding, favoritos, historial, comunidad, configuración
adminController	/api/admin/*	Dashboard, gestión CRUD de

		usuarios/películas/análisis, historial admin
--	--	---

Backend — Servicios:

Servicio	Archivo	Responsabilidad
tmdbService	services/tmdbService.js	Consulta a TMDb API: búsqueda multi-idioma, detalles, keywords, créditos, similares
geminiService	services/geminiService.js	Construcción de prompts y llamada a Groq API para generar análisis por capas
clasificadorService	services/clasificadorService.js	Clasificación automática entretenimiento/profundo por keywords, géneros, sinopsis y director
emailService	services/emailService.js	Envío de correos: verificación, código de registro, reset password, contacto

Frontend — Páginas:

Página	Archivo HTML	Descripción
Inicio	index.html	Landing page con hero, características y CTA
Búsqueda	busqueda.html	Buscador principal, categorías, recomendaciones personalizadas
Análisis	analisis.html	Visualización del análisis con pestañas por capas, detalles y favoritos
Login	login.html	Inicio de sesión con email/contraseña o Google OAuth
Registro	registro.html	Registro con verificación por código de 6 dígitos o Google
Onboarding	onboarding.html	Selección de nivel cinéfilo y géneros favoritos
Perfil	perfil.html	Datos del usuario, favoritos, historial, edición de perfil
Admin	admin.html	Panel de administración con dashboard y gestión completa
Contacto	contacto.html	Formulario de contacto funcional con envío real

Sobre nosotros	sobre-nosotros.html	Información del equipo y proyecto
Privacidad	privacidad.html	Política de privacidad
404	404.html	Página de error personalizada
Verificar email	verificar-email.html	Confirmación de correo electrónico
Reset password	reset-password.html	Restablecimiento de contraseña

Patrones utilizados y APIs

Patrones de diseño:

- MVC (Modelo-Vista-Controlador): separación clara entre rutas, controladores, modelos y vistas.
- Service Layer: la lógica de integración con APIs externas está encapsulada en servicios independientes.
- Middleware Pipeline: Express procesa autenticación, validación y autorización como middlewares encadenados.
- Cache Pattern: análisis generados se almacenan en BD para evitar llamadas repetidas a la API de IA. Películas populares usan caché en memoria con TTL de 30 minutos.

APIs externas utilizadas:

API	Propósito	Endpoints usados
TMDB API v3	Datos de películas: búsqueda, detalles, géneros, keywords, créditos, similares	/search/multi, /movie/{id}, /tv/{id}, /discover/movie, /discover/tv
Groq API	Generación de análisis	/openai/v1/chat/completions

	cinematográficos con LLaMA 3.3 70B	
Google OAuth 2.0	Autenticación social con cuentas de Google	google-auth-library (verifyIdToken)
YouTube Data API v3	Obtención de tráilers oficiales	/search (type=video, q=trailer)

8. Pruebas Realizadas

Pruebas realizadas

Tipo de prueba	Área	Descripción	Resultado
Funcional	Autenticación	Registro con código de verificación, login, Google OAuth, recuperación de contraseña	Aprobado
Funcional	Búsqueda	Búsqueda por nombre, por género (inc. K-Drama), películas populares, multi-idioma	Aprobado
Funcional	Análisis IA	Generación de análisis entretenimiento y profundo con Groq, caché en BD	Aprobado
Funcional	Clasificación	Clasificación automática por keywords, géneros, sinopsis y director	Aprobado
Funcional	Perfil	Edición de foto, bio, géneros favoritos, nivel cinéfilo	Aprobado
Funcional	Admin	Dashboard, CRUD de usuarios/películas, regeneración de análisis	Aprobado
Funcional	Contacto	Envío de formulario de contacto con recepción en magicfilm001@gmail.com	Aprobado
Integración	APIs	Comunicación correcta con TMDb, Groq, Google OAuth, YouTube	Aprobado
Responsivo	Frontend	Visualización correcta en	Aprobado

		escritorio, tablet y móvil (375px+)	
Seguridad	Auth	Hashing bcrypt, tokens JWT con expiración, middleware de autorización	Aprobado
Rendimiento	Análisis	Generación primera vez < 20s, con caché < 3s	Aprobado

Errores conocidos

- Si la API de Groq alcanza su límite de tasa (rate limit), la generación del análisis falla temporalmente. El sistema muestra un mensaje claro al usuario y permite reintentar.
- En conexiones de red muy lentas, la búsqueda puede tardar más de lo esperado al consultar TMDb. Se muestra un indicador de carga.
- Las imágenes de perfil almacenadas en base64 (MEDIUMTEXT) pueden aumentar significativamente el tamaño de la base de datos con muchos usuarios.

9. Conclusiones

Dificultades encontradas

- Calibración del clasificador de películas: lograr que el sistema distinga correctamente entre cine de entretenimiento y cine de autor requirió múltiples iteraciones, ajustando pesos de keywords, géneros, sinopsis y directores reconocidos.
- Calidad de los prompts de IA: la calidad del análisis depende críticamente del prompt enviado a Groq. Se iteraron múltiples versiones hasta lograr respuestas consistentes, bien estructuradas y en español.
- Manejo de límites de APIs: tanto TMDb como Groq tienen límites de tasa que debieron gestionarse con caché inteligente y manejo de errores adecuado.

- Autenticación con Google OAuth: la integración con Google Sign-In requirió manejar múltiples flujos (One Tap, redirect) y la verificación de tokens en el backend.

Aprendizajes

- Integración de múltiples APIs externas en un flujo coherente (TMDB → clasificador → Groq → BD).
- Diseño de una arquitectura MVC limpia en Node.js sin frameworks de frontend.
- Implementación de un sistema de autenticación completo (JWT, bcrypt, OAuth, verificación por email, reset password).
- Construcción de prompts efectivos para modelos de lenguaje (prompt engineering).
- Diseño de interfaces responsivas con temática cinematográfica y soporte dark/light mode.
- Gestión de un proyecto de software completo desde los requisitos hasta el despliegue.

Mejoras futuras

- Sistema de comentarios y valoraciones de análisis entre usuarios.
- Soporte multi-idioma (inglés, portugués) en los análisis generados.
- Migración de imágenes de perfil de base64 a almacenamiento en la nube (S3, Cloudinary).
- Implementación de WebSockets para notificaciones en tiempo real.
- Despliegue en producción con Docker y CI/CD (Vercel + PlanetScale o Railway).
- Modo offline con Service Workers para análisis ya generados.
- Integración con más proveedores de IA para comparar calidad de análisis.

10. Glosario

Término	Descripción
---------	-------------

API	Application Programming Interface. Interfaz que permite que una aplicación se comuniquen con otra mediante solicitudes HTTP.
Backend	Parte del sistema que se ejecuta en el servidor. Procesa lógica de negocio, accede a la base de datos y se comunica con APIs externas.
bcrypt	Algoritmo de hashing adaptativo utilizado para almacenar contraseñas de forma segura.
CRUD	Create, Read, Update, Delete. Operaciones básicas de gestión de datos.
Endpoint	URL específica en una API REST que responde a solicitudes HTTP (GET, POST, PUT, DELETE).
Frontend	Parte del sistema que se ejecuta en el navegador del usuario. Interfaz visual e interactiva.
Groq	Plataforma de inferencia de IA que ejecuta modelos de lenguaje como LLaMA con alta velocidad.
JWT	JSON Web Token. Estándar para transmitir información de autenticación de forma segura entre cliente y servidor.
K-Drama	Género que engloba series y películas de drama coreano, muy popular a nivel mundial.
LLaMA	Large Language Model Meta AI. Familia de modelos de lenguaje desarrollados por Meta, utilizados a través de Groq.
MVC	Modelo-Vista-Controlador. Patrón arquitectónico que separa la lógica de datos, presentación y control.
MySQL	Sistema de gestión de bases de datos relacional de código abierto.
Node.js	Entorno de ejecución de JavaScript del lado del servidor, basado en el motor V8 de Chrome.

OAuth	Protocolo de autorización que permite a aplicaciones acceder a recursos de un usuario en un servicio externo.
Onboarding	Proceso de configuración inicial que personaliza la experiencia del usuario al registrarse.
Prompt	Instrucción de texto enviada a un modelo de IA para indicarle qué contenido debe generar.
REST	Representational State Transfer. Estilo arquitectónico para diseñar APIs web basadas en recursos y métodos HTTP.
TMDB	The Movie Database. Base de datos pública y gratuita de información de películas y series.

11. Referencias y Recursos

- Repositorio de código fuente: [URL del repositorio Git]
- TMDB API v3 — Documentación oficial: <https://developer.themoviedb.org/docs>
- Groq API — Documentación: <https://console.groq.com/docs>
- Node.js — Documentación: <https://nodejs.org/docs>

- Express.js — Documentación: <https://expressjs.com/>
- MySQL 8.0 — Documentación: <https://dev.mysql.com/doc/refman/8.0/en/>
- Google OAuth 2.0 — Guía: <https://developers.google.com/identity>
- Nodemailer — Documentación: <https://nodemailer.com/>
- Font Awesome 6.5 — Iconografía: <https://fontawesome.com/>
- SweetAlert2 — Diálogos: <https://sweetalert2.github.io/>
- Documentación Magic Film (SRS):
<https://github.com/Michael5006/Magic-Film/blob/main/docs/Documentacion.Magic-Film.pdf>